

CONFIG Y GENERACIÓN DE DATASET

```

import pandas as pd
import numpy as np
from random import choices
import matplotlib.pyplot as plt

np.random.seed(42)
N = 10000
TARIFA_KWH=0.75

fechas = pd.date_range(
    start="2026-01-01",
    end="2026-12-31",
    freq="D"
)

países = [
    "Perú",
    "Colombia",
    "México"
]

prob_países = [
    0.25,
    0.30,
    0.45
]

localidades = {

    "Perú":[
        "Lima",
        "Arequipa",
        "Cusco",
        "Piura",
        "Puno"
    ],

    "Colombia":[
        "Bogotá",
        "Medellín",
        "Cali",
        "Barranquilla",
        "Bucaramanga"
    ],

    "México":[
        "Ciudad de México",
        "Guadalajara",
        "Monterrey",
        "Mérida",
        "Tijuana"
    ]
}

temperaturas = {

    "Lima": {
        1:27,
        2:28,
        3:27,
        4:25,
        5:22,
        6:20,
        7:18,
        8:18,
        9:19,
        10:21,
        11:23,
        12:25
    },

    "Arequipa": {
        1:17,
        2:17,
        3:17,
        4:16,
        5:15,

```

```
6:14,  
7:13,  
8:14,  
9:15,  
10:16,  
11:17,  
12:17  
},
```

```
"Cusco": {  
  1:14,  
  2:14,  
  3:14,  
  4:13,  
  5:12,  
  6:11,  
  7:10,  
  8:11,  
  9:12,  
  10:13,  
  11:14,  
  12:14  
},
```

```
"Piura": {  
  1:28,  
  2:29,  
  3:29,  
  4:28,  
  5:26,  
  6:24,  
  7:23,  
  8:23,  
  9:24,  
  10:25,  
  11:27,  
  12:28  
},
```

```
"Puno": {  
  1:12,  
  2:12,  
  3:12,  
  4:11,  
  5:10,  
  6:8,  
  7:8,  
  8:9,  
  9:10,  
  10:11,  
  11:12,  
  12:12  
},
```

```
"Bogotá": {  
  1:14,  
  2:14,  
  3:14,  
  4:14,  
  5:14,  
  6:13,  
  7:13,  
  8:13,  
  9:13,  
  10:14,  
  11:14,  
  12:14  
},
```

```
"Medellín": {  
  1:22,  
  2:22,  
  3:22,  
  4:22,  
  5:22,  
  6:21,  
  7:21,  
  8:21,  
  9:21,  
  10:22,  
  11:22,
```

```
12:22
},

"Cali": {
  1:24,
  2:24,
  3:24,
  4:24,
  5:24,
  6:23,
  7:23,
  8:23,
  9:23,
  10:24,
  11:24,
  12:24
},

"Barranquilla": {
  1:28,
  2:28,
  3:29,
  4:29,
  5:29,
  6:29,
  7:29,
  8:29,
  9:29,
  10:29,
  11:29,
  12:28
},

"Bucaramanga": {
  1:23,
  2:23,
  3:23,
  4:23,
  5:23,
  6:22,
  7:22,
  8:22,
  9:22,
  10:23,
  11:23,
  12:23
},

"Ciudad de México": {
  1:13,
  2:14,
  3:16,
  4:18,
  5:19,
  6:19,
  7:18,
  8:18,
  9:18,
  10:17,
  11:15,
  12:14
},

"Guadalajara": {
  1:19,
  2:20,
  3:22,
  4:24,
  5:26,
  6:25,
  7:24,
  8:24,
  9:24,
  10:23,
  11:21,
  12:19
},

"Monterrey": {
  1:15,
  2:18,
```

```

        3:22,
        4:26,
        5:29,
        6:31,
        7:32,
        8:31,
        9:29,
        10:25,
        11:20,
        12:16
    },

    "Mérida": {
        1:24,
        2:25,
        3:27,
        4:29,
        5:30,
        6:31,
        7:31,
        8:31,
        9:30,
        10:29,
        11:27,
        12:25
    },

    "Tijuana": {
        1:14,
        2:15,
        3:16,
        4:17,
        5:19,
        6:21,
        7:23,
        8:24,
        9:23,
        10:21,
        11:18,
        12:15
    }
}

tipos_inmueble = [
    "Casa",
    "Departamento",
    "Local"
]

prob_tipo = [
    0.55,
    0.30,
    0.15
]

frecuencias = [
    "Baja",
    "Media",
    "Alta"
]

prob_frecuencia = [
    0.25,
    0.45,
    0.30
]

prob_horario_pico = {
    "Casa":0.70,
    "Departamento":0.60,
    "Local":0.85
}

def generar_habitantes(tipo):
    if tipo=="Casa":
        return np.random.choice([1,2,3,4,5,6,7,8,10],p=[0.05,0.10,0.15,0.20,0.20,0.15,0.08,0.05,0.02])
    elif tipo=="Departamento":
        return np.random.choice([1,2,3,4,5,6,8],p=[0.15,0.30,0.30,0.15,0.06,0.03,0.01])
    else:

```

```

        return np.random.choice([1,2,3,5,8,10,15,20],p=[0.05,0.10,0.15,0.20,0.20,0.15,0.10,0.05])

def generar_equipos(tipo):
    if tipo=="Casa":
        return round(np.random.triangular(4,10,25))
    elif tipo=="Departamento":
        return round(np.random.triangular(3,7,15))
    else:
        return round(np.random.triangular(8,15,35))

def generar_horas_consumo(frecuencia):
    if frecuencia=="Baja":
        return np.random.randint(1,7)
    elif frecuencia=="Media":
        return np.random.randint(4,11)
    else:
        return np.random.randint(8,16)

```

CREACION DE 10000 REGISTROS

```

datos = []

for i in range(N):

    # Fecha de registro
    fecha = pd.Timestamp(np.random.choice(fechas))

    # País
    pais = np.random.choice(paises,p=prob_paises)

    # Localidad según país
    localidad = np.random.choice(
        localidades[pais]
    )

    # Temperatura según localidad y mes
    temperatura = temperaturas[localidad][fecha.month]

    temperatura = round(temperatura + np.random.uniform(-1,1),1)

    # Tipo de inmueble
    tipo = np.random.choice(
        tipos_inmueble,
        p=prob_tipo)

    # Variables relacionadas al inmueble

    habitantes = generar_habitantes(tipo)
    equipos = generar_equipos(tipo)

    # Frecuencia de uso
    frecuencia = np.random.choice(
        frecuencias,
        p=prob_frecuencia)

    # Horas de alto consumo
    horas_alto_consumo = generar_horas_consumo(
        frecuencia)

    # Uso horario pico

    uso_horario_pico = np.random.choice(
        [True, False],
        p=[prob_horario_pico[tipo],1 - prob_horario_pico[tipo]])

    # CALCULO DEL CONSUMO MENSUAL kWh

    # Consumo base según inmueble

    if tipo == "Casa":

```

```
consumo_base = np.random.triangular(120,220,350)

elif tipo == "Departamento":
    consumo_base = np.random.triangular(80,160,280)

else:

    consumo_base = np.random.triangular(200,400,700)

# Factor habitantes
factor_habitantes = 1 + ((habitantes - 2) * 0.05)

if factor_habitantes < 0.85:
    factor_habitantes = 0.85

# Factor cantidad de equipos
factor_equipos = 1 + ((equipos - 5) * 0.03)

# Factor frecuencia
if frecuencia == "Baja":
    factor_frecuencia = 0.85

elif frecuencia == "Media":
    factor_frecuencia = 1

else:
    factor_frecuencia = 1.20

# Factor horas de alto consumo
factor_horas = 1 + ((horas_alto_consumo - 5) * 0.03)

# Factor temperatura
if temperatura >= 28:
    factor_temperatura = 1.15

else:
    factor_temperatura = 1

# Variación natural del consumo
variacion = np.random.uniform(0.90,1.05)

consumo_kwh = (
    consumo_base
    *
    factor_habitantes
    *
    factor_equipos
    *
    factor_frecuencia
    *
    factor_horas
    *
    factor_temperatura
    *
    variacion)

consumo_kwh = round(consumo_kwh,2)

# CLASIFICACIÓN ENERGÉTICA

if consumo_kwh < 250:
    categoria = "Eficiente"

elif consumo_kwh < 450:
    categoria = "Moderado"

else:
    categoria = "Ineficiente"

# COSTO ESTIMADO
```

```
costo_estimado = round(consumo_kwh * TARIFA_KWH,2)

# Guardar registro
datos.append([

    fecha,
    pais,
    localidad,
    temperatura,
    tipo,
    habitantes,
    equipos,
    frecuencia,
    horas_alto_consumo,
    uso_horario_pico,
    consumo_kwh,
    categoria,
    costo_estimado])

df = pd.DataFrame(
    datos,columns=["fecha_registro",
                  "pais",
                  "localidad",
                  "temperatura_ambiente",
                  "tipo_inmueble",
                  "numero_habitantes",
                  "cantidad_equipos",
                  "frecuencia_uso",
                  "horas_alto_consumo",
                  "uso_horario_pico",
                  "consumo_kwh",
                  "categoria",
                  "costo_estimado"
    ]
)
```

df.head()

	fecha_registro	pais	localidad	temperatura_ambiente	tipo_inmueble	numero_habitantes	cantidad_equipos	frecuencia
0	2026-04-13	México	Monterrey	26.6	Departamento	2	5	1
1	2026-09-15	México	Guadalajara	23.4	Casa	4	13	1
2	2026-07-07	Colombia	Cali	22.8	Local	5	26	1
3	2026-08-30	Colombia	Medellín	20.2	Departamento	2	5	1
4	2026-01-02	Colombia	Medellín	22.1	Casa	6	12	1

Próximos pasos:

[Generar código con df](#)[New interactive sheet](#)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 13 columns):
#   Column                      Non-Null Count  Dtype
---  ---
0   fecha_registro              10000 non-null  datetime64[ns]
1   pais                        10000 non-null  object
2   localidad                   10000 non-null  object
3   temperatura_ambiente        10000 non-null  float64
4   tipo_inmueble               10000 non-null  object
5   numero_habitantes           10000 non-null  int64
6   cantidad_equipos            10000 non-null  int64
7   frecuencia_uso              10000 non-null  object
8   horas_alto_consumo          10000 non-null  int64
9   uso_horario_pico            10000 non-null  bool
10  consumo_kwh                 10000 non-null  float64
11  categoria                   10000 non-null  object
12  costo_estimado              10000 non-null  float64
dtypes: bool(1), datetime64[ns](1), float64(3), int64(3), object(5)
memory usage: 947.4+ KB
```

df.describe()

	fecha_registro	temperatura_ambiente	numero_habitantes	cantidad_equipos	horas_alto_consumo	consumo_kwh	costo_es
count	10000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	2026-07-02 07:45:33.120000	20.999750	4.478700	12.538000	7.44980	395.221959	296.4
min	2026-01-01 00:00:00	7.000000	1.000000	3.000000	1.00000	67.200000	50.4
25%	2026-04-03 00:00:00	15.800000	3.000000	8.000000	5.00000	226.165000	169.0
50%	2026-07-03 00:00:00	21.900000	4.000000	11.000000	7.00000	312.375000	234.2
75%	2026-09-30	24.000000	6.000000	16.000000	10.00000	459.500000	342.4

df.isnull().sum()	
	0
fecha_registro	0
pais	0
localidad	0
temperatura_ambiente	0
tipo_inmueble	0
numero_habitantes	0
cantidad_equipos	0
frecuencia_uso	0
horas_alto_consumo	0
uso_horario_pico	0
consumo_kwh	0
categoria	0
costo_estimado	0
dtype: int64	

Todos los atributos presentan 0 valores nulos

df.duplicated().sum()
np.int64(0)

Todos los atributos presentan 0 valores duplicados

Identificando errores en el dataset creado

df.describe()							
	fecha_registro	temperatura_ambiente	numero_habitantes	cantidad_equipos	horas_alto_consumo	consumo_kwh	costo_es
count	10000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	2026-07-02 07:45:33.120000	20.999750	4.478700	12.538000	7.44980	395.221959	296.400000
min	2026-01-01 00:00:00	7.000000	1.000000	3.000000	1.00000	67.200000	50.400000
25%	2026-04-03 00:00:00	15.800000	3.000000	8.000000	5.00000	226.165000	169.000000
50%	2026-07-03 00:00:00	21.900000	4.000000	11.000000	7.00000	312.375000	234.200000
75%	2026-09-30 00:00:00	24.000000	6.000000	16.000000	10.00000	459.500000	342.400000

El dataset contiene 10 000 registros con variables numéricas que presentan rangos coherentes con las reglas de negocio definidas. La temperatura ambiente varía entre 7 °C y 33 °C, representando las diferentes condiciones climáticas de Perú, Colombia y México. El

consumo mensual oscila entre 67.20 kWh y 2462.01 kWh, donde los valores más altos corresponden a escenarios de alto consumo que serán analizados posteriormente para verificar su consistencia.

df[df["numero_habitantes"] == 20]								
	fecha_registro	pais	localidad	temperatura_ambiente	tipo_inmueble	numero_habitantes	cantidad_equipos	frecu
347	2026-05-02	México	Tijuana	18.9	Local	20	18	
492	2026-08-07	Colombia	Bucaramanga	21.8	Local	20	25	
551	2026-08-20	Perú	Arequipa	13.4	Local	20	17	
615	2026-10-08	México	Mérida	29.0	Local	20	14	
900	2026-11-13	Colombia	Medellín	21.1	Local	20	11	
...	...	...	...	...	...	...	...	...
9229	2026-08-28	Colombia	Barranquilla	28.4	Local	20	16	
9438	2026-03-28	México	Tijuana	16.9	Local	20	17	
9582	2026-11-23	Colombia	Barranquilla	29.4	Local	20	18	
9669	2026-06-12	Colombia	Bogotá	12.6	Local	20	21	
9951	2026-01-17	Colombia	Bucaramanga	23.1	Local	20	25	
81 rows x 13 columns								

Se verificaron los registros con el número máximo de habitantes (20 personas) para validar que correspondieran a establecimientos comerciales. Se comprobó que todos los casos pertenecen al tipo de inmueble Local, por lo que no existen inconsistencias como viviendas con un número poco realista de habitantes. Esto confirma que las reglas utilizadas para la simulación fueron aplicadas correctamente.

df.nlargest(10,"consumo_kwh")								
	fecha_registro	pais	localidad	temperatura_ambiente	tipo_inmueble	numero_habitantes	cantidad_equipos	frecuer
3946	2026-06-28	México	Mérida	30.8	Local	20	22	
4394	2026-03-19	Colombia	Barranquilla	28.4	Local	10	28	
4046	2026-03-14	Perú	Cusco	14.4	Local	15	25	
4097	2026-09-20	México	Ciudad de México	17.1	Local	20	25	
2719	2026-09-05	México	Tijuana	23.8	Local	15	25	
4107	2026-09-13	Colombia	Barranquilla	29.5	Local	3	30	
3148	2026-09-15	Perú	Piura	23.4	Local	20	25	
1049	2026-10-15	Colombia	Barranquilla	28.8	Local	15	23	
8934	2026-05-14	México	Mérida	31.0	Local	10	24	
6494	2026-01-23	Colombia	Bogotá	14.6	Local	20	29	

Los registros con mayor consumo energético corresponden principalmente a locales comerciales con una alta cantidad de equipos, frecuencia de uso alta y un elevado número de horas de consumo. Estos valores representan escenarios extremos, pero coherentes, por lo que se consideran outliers válidos y no errores en los datos.

df["categoria"].value_counts()	
	count
categoria	
Moderado	4185
Eficiente	3229
Ineficiente	2586
dtype: int64	

La distribución de las categorías energéticas presenta una proporción adecuada entre las tres clases (Eficiente, Moderado e Ineficiente). Esto reduce el riesgo de desbalance de clases durante el entrenamiento de los modelos de clasificación y favorece un aprendizaje más representativo.

```
df.to_csv(
    "consumo_energia.csv",
    index=False
)
```

Una vez verificada la consistencia de los datos generados, se exporta el dataset a formato CSV para facilitar su reutilización durante el análisis exploratorio, el entrenamiento del modelo y la integración con los demás módulos del proyecto.

EDA (Exploratory Data Analysis)

```
df.head()
```

	fecha_registro	pais	localidad	temperatura_ambiente	tipo_inmueble	numero_habitantes	cantidad_equipos	frecuencia_uso
0	2026-04-13	México	Monterrey	26.6	Departamento	2	5	1
1	2026-09-15	México	Guadalajara	23.4	Casa	4	13	1
2	2026-07-07	Colombia	Cali	22.8	Local	5	26	1
3	2026-08-30	Colombia	Medellín	20.2	Departamento	2	5	1
4	2026-01-02	Colombia	Medellín	22.1	Casa	6	12	1

Próximos pasos: [Generar código con df](#) [New interactive sheet](#)

Se visualizan los primeros registros del conjunto de datos para verificar que las variables fueron generadas correctamente y presentan el formato esperado.

```
df.shape
```

(10000, 13)

El conjunto de datos está compuesto por 10 000 registros y 13 variables, las cuales contienen información relacionada con el consumo energético, características del inmueble y variables ambientales.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 13 columns):
#   Column                      Non-Null Count  Dtype
---  -
0   fecha_registro              10000 non-null  datetime64[ns]
1   pais                        10000 non-null  object
2   localidad                   10000 non-null  object
3   temperatura_ambiente        10000 non-null  float64
4   tipo_inmueble               10000 non-null  object
5   numero_habitantes           10000 non-null  int64
6   cantidad_equipos            10000 non-null  int64
7   frecuencia_uso              10000 non-null  object
8   horas_alto_consumo          10000 non-null  int64
9   uso_horario_pico            10000 non-null  bool
10  consumo_kwh                 10000 non-null  float64
11  categoria                   10000 non-null  object
12  costo_estimado              10000 non-null  float64
dtypes: bool(1), datetime64[ns](1), float64(3), int64(3), object(5)
memory usage: 947.4+ KB
```

Se verifican los tipos de datos de cada variable y la existencia de valores nulos. Todas las columnas presentan el tipo de dato esperado y no se identifican registros faltantes.

```
df["pais"].value_counts()
```

	count
pais	
México	4525
Colombia	3036
Perú	2439

dtype: int64

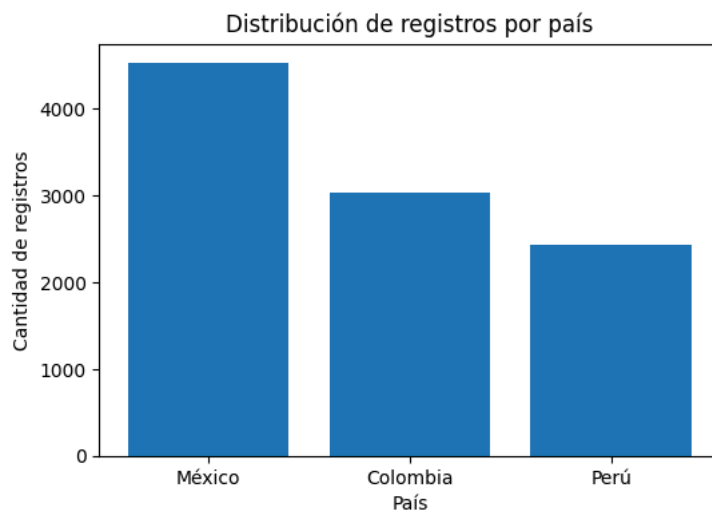
```
import matplotlib.pyplot as plt

conteo_paises = df["pais"].value_counts()

plt.figure(figsize=(6,4))
plt.bar(conteo_paises.index, conteo_paises.values)

plt.title("Distribución de registros por país")
plt.xlabel("País")
plt.ylabel("Cantidad de registros")

plt.show()
```



```
(df["pais"].value_counts(normalize=True) * 100).round(2)
```

	proportion
pais	
México	45.25
Colombia	30.36
Perú	24.39

dtype: float64

El gráfico muestra que México concentra la mayor cantidad de registros, seguido de Colombia y Perú. Esta distribución permite analizar el consumo energético considerando diferentes contextos geográficos y climáticos, aportando diversidad al conjunto de datos.

```
df["localidad"].value_counts()
```

localidad	count
Ciudad de México	940
Mérida	913
Tijuana	912
Guadalajara	882
Monterrey	878
Medellín	635
Barranquilla	611
Bogotá	606
Bucaramanga	606
Cali	578
Lima	504
Arequipa	500
Cusco	482
Piura	477
Puno	476

dtype: int64

```
import matplotlib.pyplot as plt

conteo_localidad = df["localidad"].value_counts()

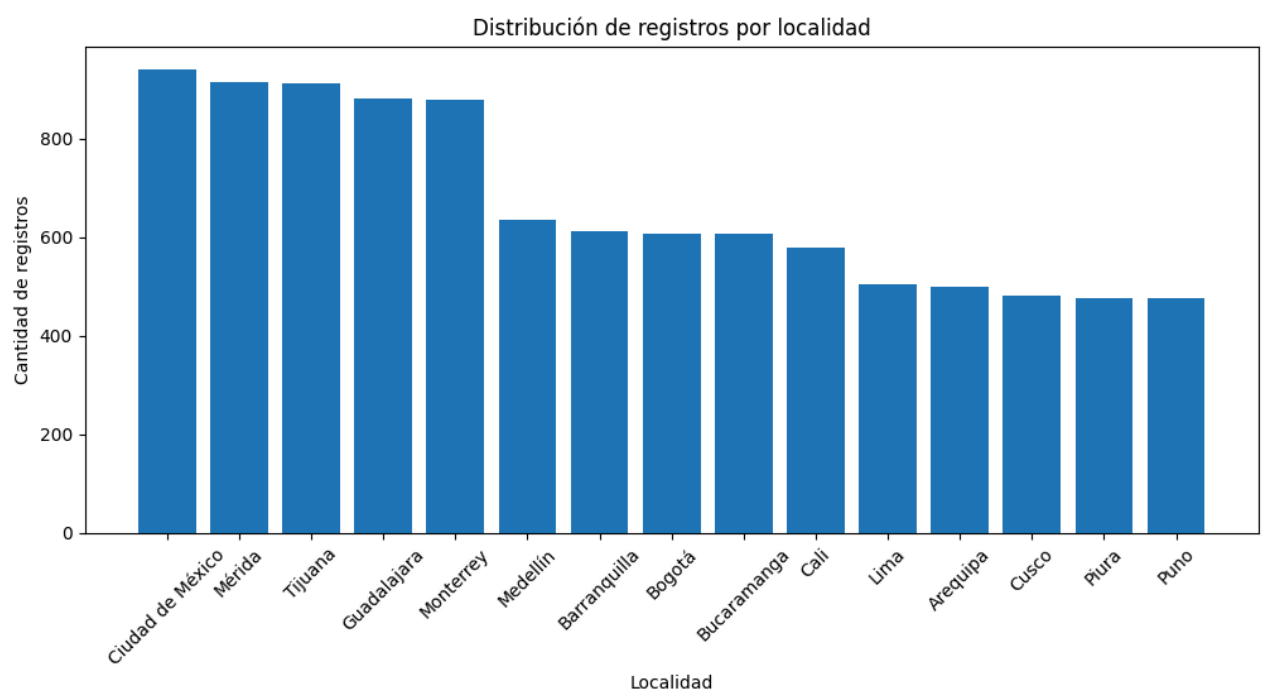
plt.figure(figsize=(12,5))

plt.bar(
    conteo_localidad.index,
    conteo_localidad.values
)

plt.title("Distribución de registros por localidad")
plt.xlabel("Localidad")
plt.ylabel("Cantidad de registros")

plt.xticks(rotation=45)

plt.show()
```



Se aprecia una distribución equilibrada entre las localidades pertenecientes a un mismo país. Asimismo, las ciudades mexicanas presentan un mayor número de registros debido a que México concentra la mayor participación dentro del conjunto de datos.

```
(df["tipo_inmueble"].value_counts(normalize=True) * 100).round(2)
```

	proportion
tipo_inmueble	
Casa	54.07
Departamento	30.38
Local	15.55

dtype: float64

```
conteo_tipo = df["tipo_inmueble"].value_counts()

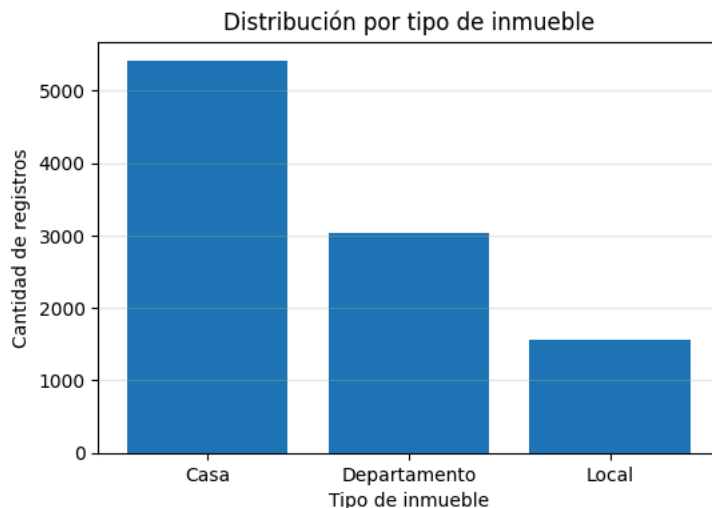
plt.figure(figsize=(6,4))

plt.bar(
    conteo_tipo.index,
    conteo_tipo.values
)

plt.title("Distribución por tipo de inmueble")
plt.xlabel("Tipo de inmueble")
plt.ylabel("Cantidad de registros")

plt.grid(axis="y", alpha=0.3)

plt.show()
```



Se observa que las viviendas tipo Casa representan la mayor proporción del conjunto de datos, seguidas por los Departamentos y los Locales. Esta distribución indica que el dataset está compuesto principalmente por inmuebles residenciales, mientras que los establecimientos comerciales constituyen una menor proporción de los registros analizados.

```
(df["categoria"].value_counts(normalize=True) * 100).round(2)
```

	proportion
categoria	
Moderado	41.85
Eficiente	32.29
Ineficiente	25.86

dtype: float64

```
conteo_categoria = df["categoria"].value_counts()

plt.figure(figsize=(6,4))

plt.bar(
```

```

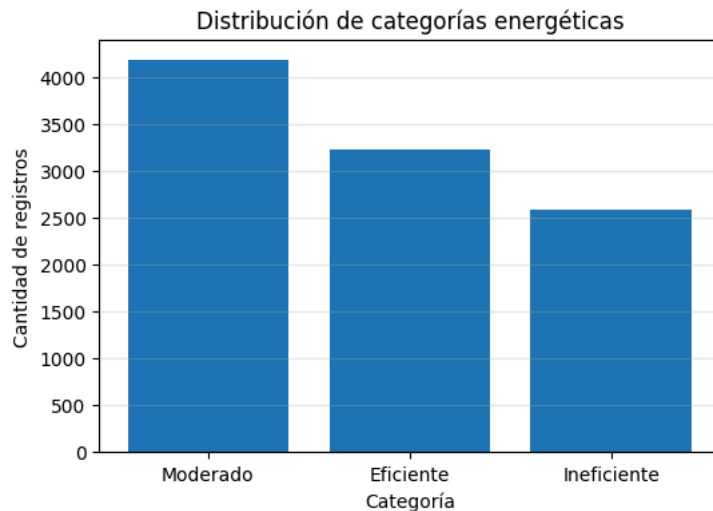
    conteo_categoria.index,
    conteo_categoria.values
)

plt.title("Distribución de categorías energéticas")
plt.xlabel("Categoría")
plt.ylabel("Cantidad de registros")

plt.grid(axis="y", alpha=0.3)

plt.show()

```



Se observa que la categoría Moderado concentra la mayor cantidad de registros, seguida por Eficiente e Ineficiente. La distribución es relativamente equilibrada entre las tres clases, lo que favorece el entrenamiento de modelos de clasificación al contar con una representación adecuada de cada perfil de consumo energético.

```

consumo_tipo = df.groupby("tipo_inmueble")["consumo_kwh"].mean().sort_values(ascending=False)

consumo_tipo

```

tipo_inmueble	consumo_kwh
Local	877.573910
Casa	355.752547
Departamento	218.577400

dtype: float64

```

plt.figure(figsize=(10,6))

barras = plt.bar(
    consumo_tipo.index,
    consumo_tipo.values
)

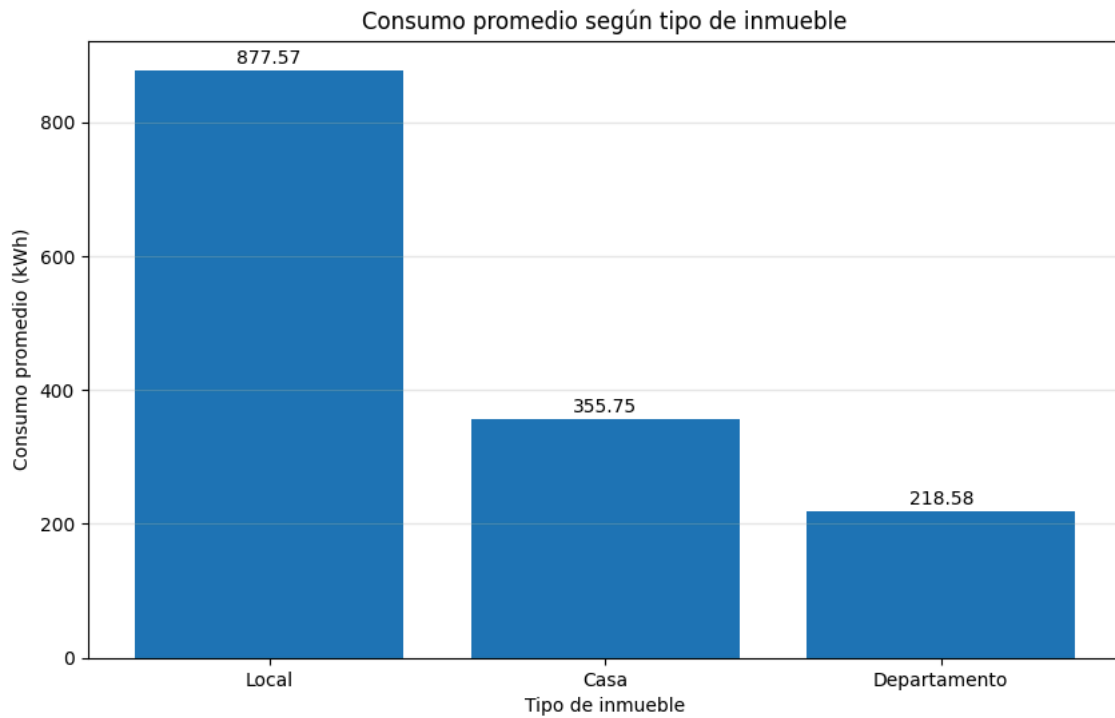
plt.title("Consumo promedio según tipo de inmueble")
plt.xlabel("Tipo de inmueble")
plt.ylabel("Consumo promedio (kwh)")

# Mostrar el valor encima de cada barra
for barra in barras:
    altura = barra.get_height()
    plt.text(
        barra.get_x() + barra.get_width()/2,
        altura + 10,
        f"{altura:.2f}",
        ha="center",
        fontsize=10
    )

plt.grid(axis="y", alpha=0.3)

plt.show()

```



Los locales comerciales registran el mayor consumo promedio de energía (877.57 kWh/mes), muy por encima de las casas (355.75 kWh/mes) y los departamentos (218.58 kWh/mes). Este comportamiento evidencia que el tipo de inmueble influye significativamente en el consumo energético, ya que los establecimientos comerciales suelen operar durante más horas y disponer de una mayor cantidad de equipos eléctricos.

```
consumo_frecuencia = (  
    df.groupby("frecuencia_uso")["consumo_kwh"]  
    .mean()  
    .sort_values(ascending=False)  
)  
  
consumo_frecuencia
```

	consumo_kwh
frecuencia_uso	
Alta	514.599667
Media	379.122837
Baja	284.333960

dtype: float64

```
plt.figure(figsize=(6,4))

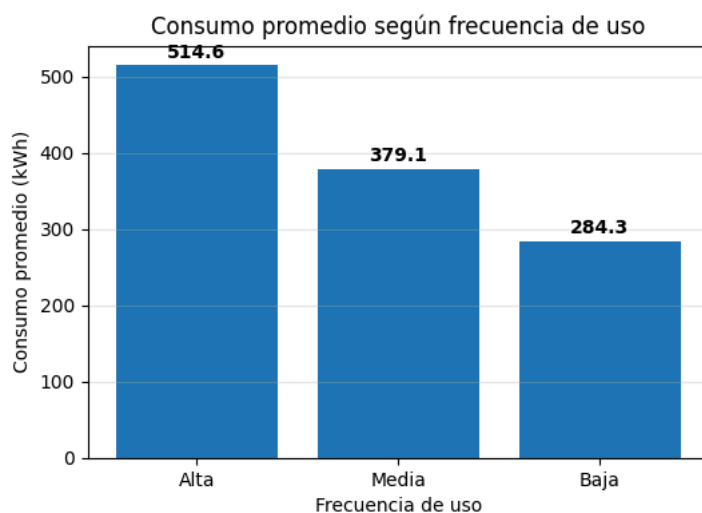
barras = plt.bar(
    consumo_frecuencia.index,
    consumo_frecuencia.values
)

plt.title("Consumo promedio según frecuencia de uso")
plt.xlabel("Frecuencia de uso")
plt.ylabel("Consumo promedio (kWh)")

for barra in barras:
    altura = barra.get_height()
    plt.text(
        barra.get_x() + barra.get_width()/2,
        altura + 10,
        f"{altura:.1f}",
        ha="center",
        fontsize=10,
        fontweight="bold"
    )

plt.grid(axis="y", alpha=0.3)

plt.show()
```



Se observa una relación directa entre la frecuencia de uso y el consumo energético mensual. Los inmuebles con una frecuencia de uso alta presentan el mayor consumo promedio (514.60 kWh/mes), seguidos por aquellos con frecuencia media (379.12 kWh/mes) y baja (284.33 kWh/mes). Este comportamiento indica que una mayor utilización de los equipos eléctricos está asociada a un incremento en la demanda energética.

```
consumo_pais = (
    df.groupby("pais")["consumo_kwh"]
    .mean()
    .sort_values(ascending=False)
)

consumo_pais
```

	consumo_kwh
pais	
Colombia	400.500418
México	395.276181
Perú	388.550882

dtype: float64

```
plt.figure(figsize=(6,4))

barras = plt.bar(
    consumo_pais.index,
    consumo_pais.values
)
```



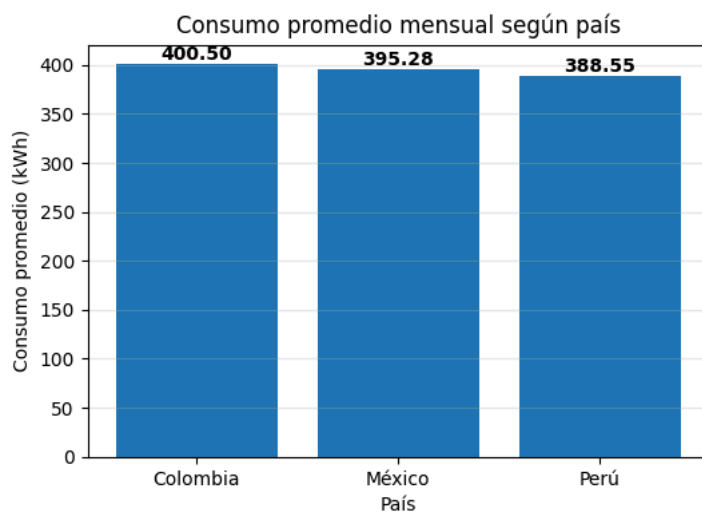
```
plt.title("Consumo promedio mensual según país")
plt.xlabel("País")
plt.ylabel("Consumo promedio (kWh)")

# Valores encima de cada barra
for barra in barras:
    altura = barra.get_height()

    plt.text(
        barra.get_x() + barra.get_width()/2,
        altura + 5,
        f"{altura:.2f}",
        ha="center",
        fontsize=10,
        fontweight="bold"
    )

plt.grid(axis="y", alpha=0.3)

plt.show()
```



El consumo promedio mensual presenta valores similares entre los países analizados. Colombia registra el mayor consumo promedio con 400.50 kWh/mes, seguido por México con 395.28 kWh/mes y Perú con 388.55 kWh/mes. La diferencia entre países es reducida, lo que indica que la ubicación geográfica por sí sola no determina el consumo energético, sino que este depende principalmente de factores como el tipo de inmueble, cantidad de equipos, frecuencia de uso y hábitos de consumo.

```
consumo_total_localidad = (
    df.groupby("localidad")["consumo_kwh"]
    .sum()
    .sort_values(ascending=False)
)

consumo_total_localidad
```

consumo_kwh	
localidad	
Mérida	386279.08
Monterrey	358925.39
Ciudad de México	353524.66
Guadalajara	345976.68
Tijuana	343918.91
Barranquilla	282084.51
Medellín	250355.08
Bogotá	231236.34
Bucaramanga	230871.92
Cali	221371.42
Lima	196097.94
Piura	193449.54
Arequipa	192767.31
Cusco	183712.25
Puno	181648.56

dtype: float64

```
plt.figure(figsize=(10,6))

barras = plt.barh(
    consumo_total_localidad.index,
    consumo_total_localidad.values
)

plt.title("Consumo energético total acumulado según localidad")
plt.xlabel("Consumo total (kWh)")
plt.ylabel("Localidad")

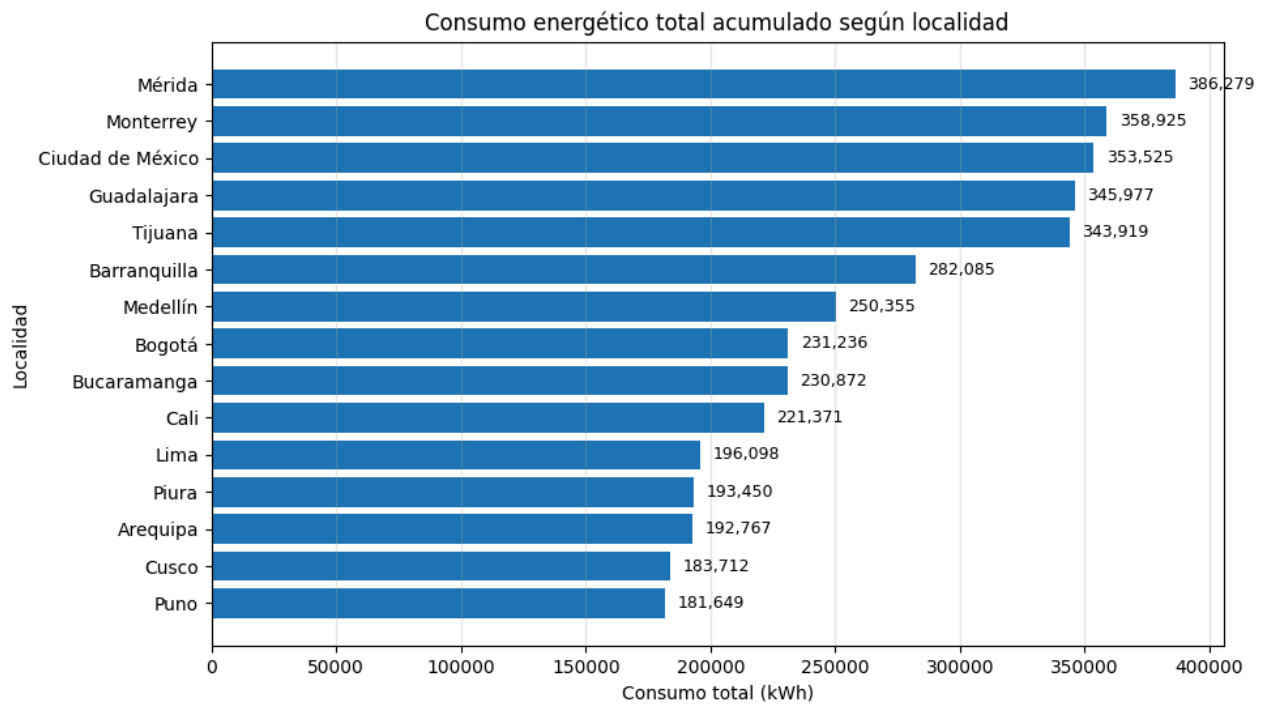
# Invertir orden del eje Y
plt.gca().invert_yaxis()

# Valores en las barras
for barra in barras:
    ancho = barra.get_width()

    plt.text(
        ancho + 5000,
        barra.get_y() + barra.get_height()/2,
        f"{ancho:,.0f}",
        va="center",
        fontsize=9
    )

plt.grid(axis="x", alpha=0.3)

plt.show()
```



El consumo energético total acumulado presenta diferencias importantes entre las localidades analizadas. Mérida concentra el mayor consumo con 386,279.08 kWh, seguida de Monterrey con 358,925.39 kWh y Ciudad de México con 353,524.66 kWh. Por otro lado, Puno registra el menor consumo acumulado con 181,648.56 kWh.

✓ CORRELACION CANTIDAD DE EQUIPOS Y CONSUMO ENERGETICO

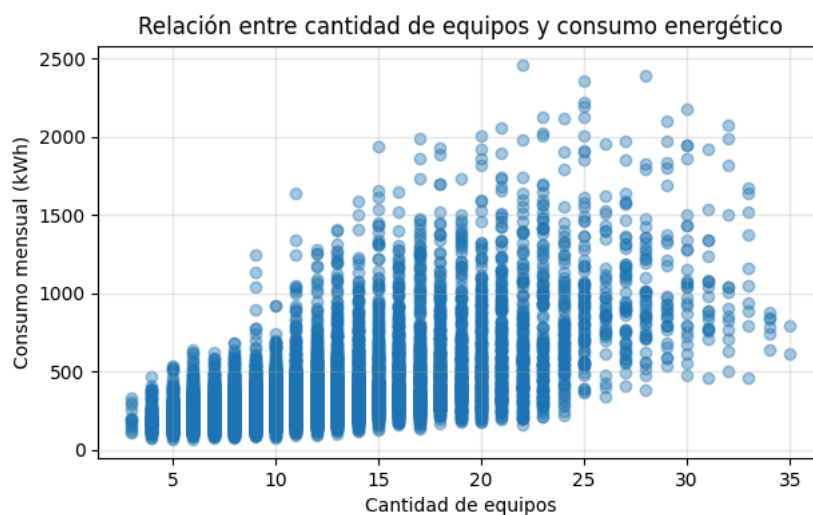
```
plt.figure(figsize=(7,4))

plt.scatter(
    df["cantidad_equipos"],
    df["consumo_kwh"],
    alpha=0.4
)

plt.title("Relación entre cantidad de equipos y consumo energético")
plt.xlabel("Cantidad de equipos")
plt.ylabel("Consumo mensual (kWh)")

plt.grid(alpha=0.3)

plt.show()
```



```
correlacion_equipos = df["cantidad_equipos"].corr(df["consumo_kwh"])

print("Correlación cantidad de equipos vs consumo:", correlacion_equipos)
```

Correlación cantidad de equipos vs consumo: 0.6149809057939047

Se observa una relación positiva moderada entre la cantidad de equipos y el consumo energético, respaldada por un coeficiente de correlación de 0.615. Esto indica que, en general, a medida que aumenta la cantidad de equipos en un inmueble, también tiende a incrementarse el consumo mensual de energía. No obstante, la dispersión de los datos evidencia que existen otros factores que también influyen en el consumo, por lo que la cantidad de equipos no es la única variable explicativa.

CORRELACION NUMERO DE HABITANTES Y CONSUMO ENERGETICO

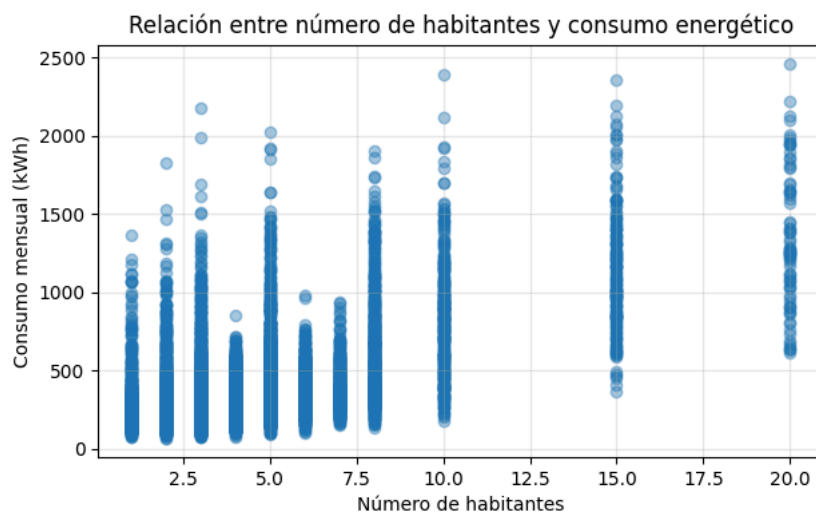
```
plt.figure(figsize=(7,4))

plt.scatter(
    df["numero_habitantes"],
    df["consumo_kwh"],
    alpha=0.4
)

plt.title("Relación entre número de habitantes y consumo energético")
plt.xlabel("Número de habitantes")
plt.ylabel("Consumo mensual (kWh)")

plt.grid(alpha=0.3)

plt.show()
```



```
correlacion_habitantes = df["numero_habitantes"].corr(df["consumo_kwh"])

print("Correlación número de habitantes vs consumo:", correlacion_habitantes)
```

Correlación número de habitantes vs consumo: 0.5824876841228638

Se observa una relación positiva moderada entre el número de habitantes y el consumo energético, respaldada por un coeficiente de correlación de 0.582. El gráfico evidencia que, conforme aumenta el número de habitantes, el consumo energético tiende a incrementarse. Sin embargo, la dispersión de los datos indica que inmuebles con una misma cantidad de habitantes pueden presentar consumos diferentes, lo que sugiere que el consumo también depende de otros factores, como la cantidad de equipos, las horas de alto consumo y la temperatura ambiente.

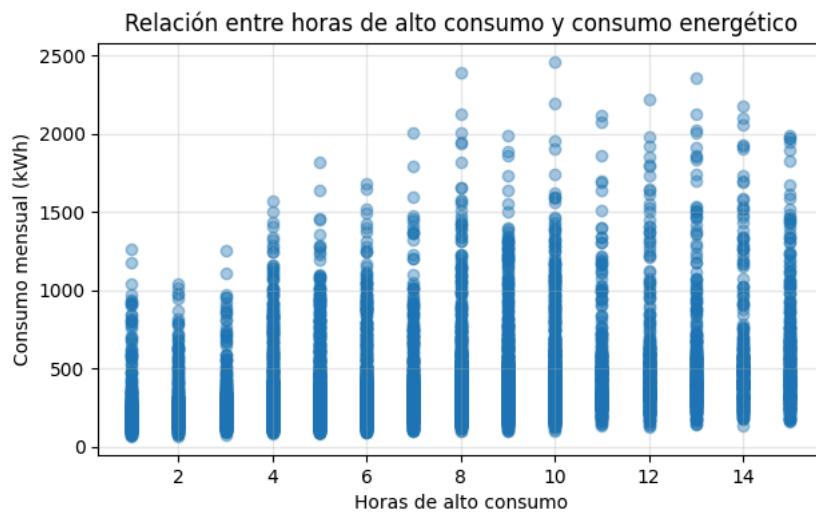
```
plt.figure(figsize=(7,4))

plt.scatter(
    df["horas_alto_consumo"],
    df["consumo_kwh"],
    alpha=0.4
)

plt.title("Relación entre horas de alto consumo y consumo energético")
plt.xlabel("Horas de alto consumo")
plt.ylabel("Consumo mensual (kWh)")

plt.grid(alpha=0.3)

plt.show()
```



```

correlacion_horas = df["horas_alto_consumo"].corr(df["consumo_kwh"])

print("Correlación horas de alto consumo vs consumo:", correlacion_horas)

Correlación horas de alto consumo vs consumo: 0.30891645824907454

```

Se observa una relación positiva débil entre las horas de alto consumo y el consumo energético, evidenciada por un coeficiente de correlación de 0.309. Aunque el gráfico muestra una ligera tendencia ascendente, la dispersión de los datos indica que el número de horas de alto consumo no explica por sí solo las variaciones en el consumo energético. Esto sugiere que el consumo depende de la interacción de múltiples variables, como la cantidad de equipos, el número de habitantes y otras características del inmueble.

Haz doble clic (o ingresa) para editar

```

plt.figure(figsize=(7,4))

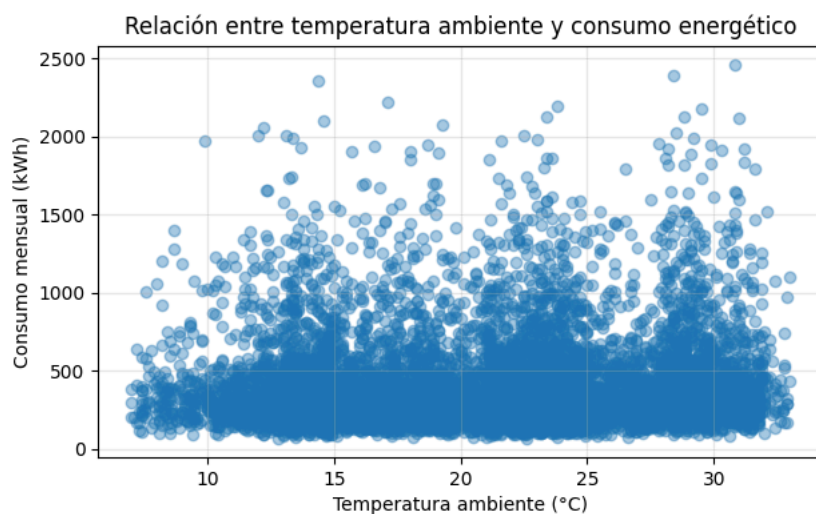
plt.scatter(
    df["temperatura_ambiente"],
    df["consumo_kwh"],
    alpha=0.4
)

plt.title("Relación entre temperatura ambiente y consumo energético")
plt.xlabel("Temperatura ambiente (°C)")
plt.ylabel("Consumo mensual (kWh)")

plt.grid(alpha=0.3)

plt.show()

```



```

correlacion_temperatura = df["temperatura_ambiente"].corr(df["consumo_kwh"])

print("Correlación temperatura vs consumo:", correlacion_temperatura)

Correlación temperatura vs consumo: 0.07081742292482407

```

Se observa una relación positiva muy débil entre la temperatura ambiente y el consumo energético, con un coeficiente de correlación de 0.071. La dispersión de los datos evidencia que la temperatura, por sí sola, no presenta una influencia significativa sobre el consumo energético en el conjunto de datos analizado. Esto sugiere que otras variables, como la cantidad de equipos y el número de habitantes, tienen un mayor impacto en el comportamiento del consumo.

```
corr = df[  
[
```